El conjunto / mapa de objetos (Object set/map): SplObjectStorage

Caso práctico

Juan está programando las alertas de stock de la tienda: cuando un producto vuelve a estar disponible, hay que avisar a los clientes que lo pidieron y al panel del almacén, **sin avisar dos veces al mismo**. Su primera idea es un array con los clientes como clave... y PHP le responde `Illegal offset type`: **los arrays no admiten objetos como clave**. **María** le presenta `SplObjectStorage`, que es justo eso: un **conjunto de objetos** y un **mapa objeto → dato**.

1.- Qué es un conjunto / mapa de objetos

- Un **conjunto** (*set*) es una colección **sin repetidos** en la que lo importante es saber si un elemento **está o no** (y hacer uniones, intersecciones...).
- Un **mapa** (*map*, diccionario) asocia cada **clave** a un **valor**.

Con los arrays de PHP se pueden hacer conjuntos y mapas **de enteros o textos**, pero no de **objetos**. `SplObjectStorage` lo resuelve: cada objeto puede estar como mucho una vez y, opcionalmente, llevar un dato asociado.

Operación	Método	Coste
Añadir (con un dato opcional)	`attach($obj, $dato)` o `$s[$obj] = $dato`	O(1)
Quitar	`detach($obj)` o `unset($s[$obj])`	O(1)
¿Está?	`contains($obj)` o `isset($s[$obj])`	O(1)
Leer el dato	`$s[$obj]`	O(1)
Tamaño	`count($s)`	O(1)
Unión / diferencia / intersección	`addAll($otro)` / `removeAll($otro)` / `removeAllExcept($otro)`	O(n)

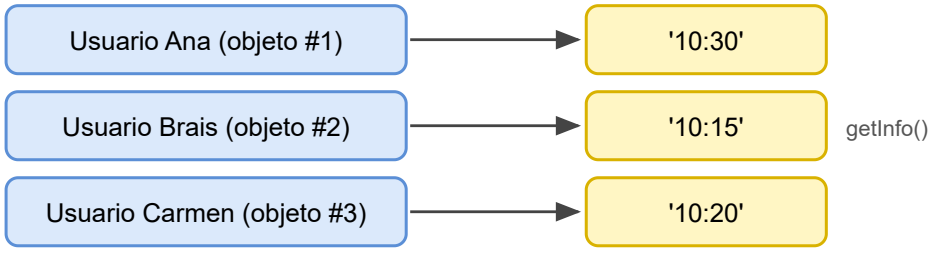


Figura 1. SplObjectStorage como mapa: cada objeto (clave) lleva su dato

2.- La clase SplObjectStorage

```php
<?php
$conectados = new SplObjectStorage();
$conectados->attach($ana);
$conectados->attach($brais);
$conectados->attach($ana);                // ya estaba: no se duplica
echo count($conectados);                  // 2
var_dump($conectados->contains($carmen)); // false

$horaConexion = new SplObjectStorage();   // como mapa
$horaConexion[$ana] = '10:02';
$horaConexion[$ana] = '10:30';            // se actualiza el dato
echo $horaConexion[$ana];                 // "10:30"
```

Leer el dato de un objeto que no está lanza `UnexpectedValueException` ("Object not found"): compruébalo antes con `isset()` o `contains()`.

Recorrer. En `foreach`, la **clave** es un número de orden (0, 1, 2...) y el **valor es el objeto**. El dato asociado al objeto actual se lee con `getInfo()` y se cambia con `setInfo()`:

```php
<?php
foreach ($horaConexion as $i => $usuario) {
    echo "$usuario->nombre desde las " . $horaConexion->getInfo(), PHP_EOL;
}
```

Debes conocer

No borres mientras recorres. Si haces `detach()` del objeto actual dentro de un `foreach` sobre el mismo `SplObjectStorage`, el recorrido se desplaza y **se salta el siguiente** elemento. Recorre una **copia** (`foreach (clone $s as $obj)`) y borra del original, o guarda los que hay que borrar en otro conjunto y quítalos después con `removeAll()`. Lo mismo ocurre si un observador se da de baja dentro de `update()` (apartado 4).

3.- ¿Cuándo son "el mismo" dos objetos? getHash()

Por defecto, `SplObjectStorage` identifica cada objeto por su **identidad** (como `===`): dos objetos distintos con los mismos datos cuentan como **dos**. Si quieres otro criterio (por ejemplo, dos `Usuario` con el mismo email son el mismo), hereda y redefine `getHash()`, que devuelve el texto que identifica al objeto:

```php
<?php
final class UsuariosPorEmail extends SplObjectStorage
{
    public function getHash(object $usuario): string
    {
        return mb_strtolower($usuario->email);
    }
}
```

Las operaciones de conjuntos (`addAll`, `removeAll`, `removeAllExcept`) modifican el propio conjunto: haz antes una copia con `clone` si quieres conservar el original.

```php
<?php
$ambos = clone $robotica;
$ambos->removeAllExcept($teatro);        // intersección: los que van a los dos talleres
$soloRobotica = clone $robotica;
$soloRobotica->removeAll($teatro);       // diferencia
```

Para saber más

WeakMap (PHP 8.0) es otro mapa objeto → dato, pero **"débil"**: no impide que el objeto se libere. Cuando nadie más usa el objeto, su entrada desaparece sola. Es ideal para **cachés** de datos calculados sobre objetos. `SplObjectStorage`, en cambio, mantiene vivos los objetos que contiene y tiene operaciones de conjuntos. Cuidado: una variable que todavía apunte al objeto (por ejemplo, la de un `foreach` anterior) también lo mantiene vivo.

4.- El patrón Observador: SplSubject y SplObserver

La SPL incluye dos **interfaces** para el patrón de diseño **Observador**: un **sujeto** (`SplSubject`: `attach`, `detach`, `notify`) avisa a sus **observadores** (`SplObserver`: `update`) cuando cambia algo. El sujeto guarda a sus observadores en un `SplObjectStorage`, así nunca avisa dos veces al mismo, y no necesita saber qué hace cada uno: se pueden añadir observadores nuevos sin tocar la clase del sujeto.

```php
<?php
final class ProductoConStock implements SplSubject
{
    private SplObjectStorage $observadores;

    public function __construct(public readonly string $nombre, private int $stock = 0)
    {
        $this->observadores = new SplObjectStorage();
    }

    public function attach(SplObserver $o): void
    {
        $this->observadores->attach($o);
    }

    public function detach(SplObserver $o): void
    {
        $this->observadores->detach($o);
    }

    public function notify(): void
    {
        foreach (clone $this->observadores as $o) {   // copia: un observador puede darse de
baja
            $o->update($this);
        }
    }

    public function reponer(int $unidades): void
    {
        $this->stock += $unidades;
        $this->notify();
    }

    public function stock(): int
    {
        return $this->stock;
    }
}
final class PanelAlmacen implements SplObserver
{
    public function update(SplSubject $producto): void
    {
        echo "{$producto->nombre}: {$producto->stock()} unidades", PHP_EOL;
    }
}
```

5.- Aplicaciones típicas

Problema	Cómo
Usuarios conectados, sesiones abiertas	Conjunto de objetos (`attach` / `detach` / `contains`)
Carrito objeto Producto → cantidad	Mapa (`$s[$producto] = $cantidad`)
Patrón Observador (alertas, eventos, boletines)	`SplSubject` + `SplObserver` + `SplObjectStorage`
Redes de seguidores, grafos de objetos	Cada objeto con un `SplObjectStorage` de sus vecinos
Unión, intersección y diferencia de grupos	`addAll`, `removeAllExcept`, `removeAll`
Cachés de datos asociados a objetos	`WeakMap`

6.- Ejercicio resuelto: el carrito de la tienda

Ejercicio resuelto

Programa un carrito que guarde, para cada objeto `Producto`, cuántas unidades se compran; que sume unidades si el producto ya estaba; que permita quitar productos; y que calcule el total.

```
<?php
final class Producto
{
    public function __construct(public readonly string $ref, public readonly string
$nombre, public readonly float $precio)
    {
    }
}
final class Carrito implements Countable
{
    private SplObjectStorage $lineas;            // Producto => unidades

    public function __construct()
    {
        $this->lineas = new SplObjectStorage();
    }

    public function anadir(Producto $p, int $unidades = 1): void
    {
        $actual = $this->lineas->contains($p) ? $this->lineas[$p] : 0;
        $this->lineas[$p] = $actual + $unidades;
    }

    public function quitar(Producto $p): void
    {
        $this->lineas->detach($p);
    }
```

```
    public function total(): float
    {
        $total = 0;
        foreach ($this->lineas as $producto) {
            $total += $producto->precio * $this->lineas->getInfo();
        }
        return round($total, 2);
    }

    public function count(): int
    {
        return array_sum(array_map(fn($p) => $this->lineas[$p], iterator_to_array($this-
>lineas, false)));
    }
}
$raton = new Producto('RAT-01', 'Ratón', 19.95);
$teclado = new Producto('TEC-02', 'Teclado', 49.50);
$carrito = new Carrito();
$carrito->anadir($raton, 2);
$carrito->anadir($teclado);
$carrito->anadir($raton);                       // ya estaba: ahora 3
echo count($carrito);                           // 4
echo $carrito->total();                         // 109.35
```

Observa que la clave es el **propio objeto**: al recorrer el carrito se recupera el producto entero (con su nombre y su precio), no solo una referencia en texto que habría que volver a buscar.

Autoevaluación

- ¿Se puede usar un objeto como clave de un array de PHP? **No: da `TypeError: Illegal offset type`. Para eso está `SplObjectStorage`.**
- Se hace `attach($ana)` dos veces. ¿Cuánto vale `count()`? **1: no se duplica.**
- En un `foreach` sobre un `SplObjectStorage`, ¿qué es el valor? ¿Y cómo se obtiene el dato asociado? **El objeto; el dato, con `getInfo()`.**
- ¿Qué hay que redefinir para que dos objetos distintos con el mismo email cuenten como uno? **`getHash()`.**
- ¿Por qué `notify()` recorre una copia de los observadores? **Porque si un observador se da de baja durante el recorrido, el `foreach` se saltaría al siguiente.**

7.- Ejercicios

Fichero	Contenido
`Ejercicios ObjectStorage/e01-SplObjectStorage.php`	La API: como conjunto y como mapa, recorrer con `getInfo` / `setInfo`, operaciones de conjuntos, `getHash`, `WeakMap`
`Ejercicios ObjectStorage/e02-SplObjectStorage.php`	Aplicaciones: carrito, patrón Observador con `SplSubject` / `SplObserver`, red de seguidores, caché con `WeakMap`
`Ejercicios ObjectStorage/problemas/`	`e01` y `e02`: problemas y soluciones
